

1) Explain the different memory areas managed by JVM.

- Stack memory
- Heap memory
- Method Area
- program counter
- native method stack

Sel:- Stack:-

- In Java, each thread has its own stack called the Run-time stack, created when the thread starts.
- It stores various things like method call frames, local variables, method parameters and returns address.
- The JVM only ^{performs} two operations directly on stack that is push (when a method is called) and pop (when a method execution completes).
- Every method call performed by the thread is stored in the corresponding run-time stack.

Heap Area:-

- The Heap Area is the memory block in the JVM where all objects are created and stored. It is used to allocate memory for:
- objects: instances of classes created during the execution of a program
- class interfaces: memory for class interfaces is allocated here.
- Arrays: Since Arrays are considered objects in Java, their memory is allocated in the heap as well.

Method Area:-

The Class Area is a memory block in the JVM that stores important information about classes. It includes:

- class code:- the bytecode that defines the class itself.
- variable code:- This includes static variables and runtime constants associated with the class.
- method code:- The bytecode for all the methods defined in the class including constructors.

Program Counter Register:-

Each JVM thread has its own PC register that stores the address of the JVM instruction currently being executed.

- For non-native method the PC holds address of the available JVM instruction.
- For native method, the PC register value is undefined.

Native Method Stack:-

It is also known as C stack and this area is used to store the state of native methods which are written in languages other than Java (for e.g. C or C++).

- Each thread has its own native method stack which can be fixed or dynamically sized.

→ This stack handles native method calls and execution.

Stacks vs Heap memory Allocation:-

Stacks:-

Stack refers to the process of assigning memory for local variables and function calls stack in the call stack.

It happens automatically when a function is called and is freed immediately when the function ends.

It also stores references of objects in stack.

```
import java.io.*;
```

```
class Student {
```

```
    String name
```

```
    int age
```

```
    Student (String name, int age)
```

```
    { this.name = name;
```

```
      this.age = age;
```

```
    }
```

```
}
```

```
public class Greeter {
```

```
    public static void main (String[] args)
```

```
    {
```

```
        Student s = new Student ("Ram", 20);
```

```
    }
```

```
}
```

→ Here the student object is stored in heap memory and its reference is stored in stack memory.

Heap Memory Allocation in Java:-

Heap memory play is a memory that can be requested by the user, when we create an object using new keyword, memory is allocated for that object in the heap. The size of an object depends on the fields and data types defined in the class.

It consists of three parts in Java

- 1) Young Generation
- 2) Old Generation
- 3) Meta space

→ when the new is called JVM allocates memory for that object in the heap. The garbage collector automatically deletes objects that are no longer needed.

3) Garbage collection in Java ?

A) Garbage collection in Java is an automatic memory management process that helps Java programs run efficiently.

→ Garbage collection is an automatic process that removes unused objects from heap.
working:-

- It identifies which objects are still in use (referenced) and which are not in use (unreferenced).
- It removes the objects that are unreachable (no longer referenced).
- The programmer does not need to mark objects to be deleted explicitly.
- Garbage collection is implemented within the JVM.
- We can manually make garbage collection to work by using `System.gc()`.

Two types of garbage collection activities usually happen in Java.

- Minor Garbage collection (GC)
- Major or Full Garbage collection (GC).

1) minor:-

This occurs when unreachable objects in the young generation heap memory are removed.

2) Major:-

This happens when objects that survive minor garbage collection are removed from the old generation heap memory.

it occurs in the process of garbage collection.

4) What are the different types of Garbage collectors available in Java? mention any two.

A) Types:-

- 1) Serial GC
- 2) parallel GC
- 3) concurrent mark-sweep collector
- 4) G1 (Garbage-First) collector
- 5) Z Garbage collector

→ Serial Garbage collector:-

type: Single-threaded collector
working: Uses one thread to perform all garbage collection activities (minor and major GC)

→ parallel Garbage collector (Throughput collector)

type: multi-threaded collector
working: Uses multiple threads to perform minor and major GC, reducing pause time

5) What is the role of the finalize method in Java? is it recommended to use it?

A) Role of finalize() in Java:-

→ finalize() is a method in the Object class

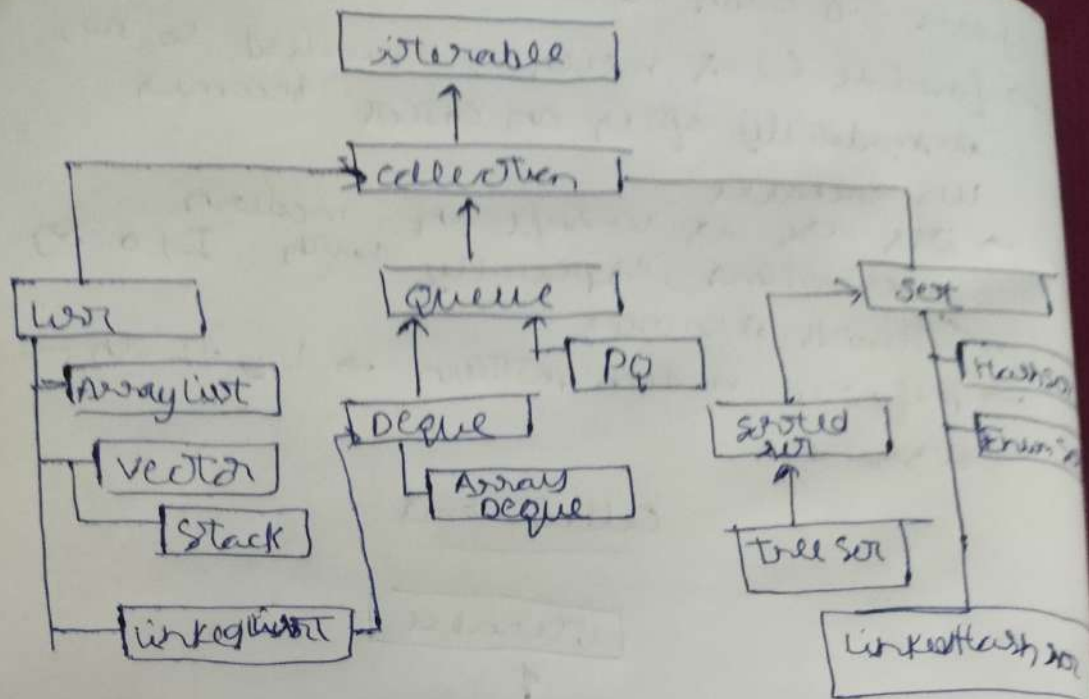
→ It is called by the GC before an object is destroyed, giving the object a

choice to clean up resources.

→ `finalize()` is never guaranteed to run immediately after an object becomes unreachable

→ its use is unsafe in modern applications, especially with I/O or network resources.

→ preferred modern pattern → try-with-resources



Differences

1) ArrayList vs linked list

Feature	ArrayList	linked list
Data Structure	Dynamic array	Doubly linked list
Access time	Fast $O(1)$	Slow $O(n)$
Insertion / Deletion	Slow $O(n)$	Fast $O(1)$
memory usage	less memory	more memory
Best for	frequent access, less insertion deletion	Frequent insertion / deletion

2) Stack vs vector

Feature Type	Stack	Vector
Inheritance	LIFO data structure Extends Vector	Dynamic array Extends AbstractList
Thread safety	Synchronized	Synchronized
Access methods	push(), pop(), peek(), poll()	get(), add(), remove(), set()
Use case	When LIFO behavior is needed	General purpose dynamic array

3) ArrayList vs Vector

Feature	ArrayList	Vector
Thread safety	Not Synchronized	Synchronized
Performance	Faster (no synchronization)	Slower (synchronization)
Growth	increased by 50% when capacity is exceeded	Doubles size when capacity exceeded
Legacy	modern, preferred	Legacy, from Java 1.0
Best For	Non-threaded scenarios	Thread-safe scenarios